

Predicting Short-Term Equity Price Direction with Machine Learning: A Case Study on SPY Using Logistic Regression

Samundra Pathak
University of Wisconsin–Green Bay

September 17, 2026

Abstract

A machine learning model was built and evaluated to predict the next day’s up or down direction of the SPDR S&P 500 ETF Trust (SPY) for 10 years of daily price data (2015-2025). A small set of technical indicators such as the daily return, short- and long-window moving averages, rolling volatility, and RSI (Relative Strength Index) were chosen to train a Logistic Regression model. A time series dataset was used, so a strict chronological train/test split was applied. The base model reached a test accuracy of 55.6%, which is lower than 57.4% for an always-predicting-up model. Class-weight balancing was applied to correct a bias of the model towards the up class. It improved the recall on down days from 0.15 to 0.95, but as a consequence, it collapsed the recall on up days to 0.03, which dropped the overall accuracy to 42.0%. This result shows us that class-weighting corrects bias during training, but cannot inherently make the chosen set of features carry a strong linearly separable signal for the next-day direction. In the final section, we discuss the implications of these findings and suggest ways to improve the base model’s accuracy.

1 Introduction

Quantitative trading strategies have historically relied on statistical data to predict the future price movement of a stock. Pioneers like Edward Thorp and James Simons were among the first to bring statistical learning, pattern recognition, and automated algorithms to quantitative finance in the 1970s and 1980s. In this case study, a simple binary classification is used to show a working pipeline. Pulling the data from Yahoo Finance, a model was built to predict if the asset’s price went higher or lower the next day.

The choice of the asset that is used here is SPY. It is an Exchange-Traded Fund (ETF) that tracks the Standard & Poor’s 500 Index (S&P 500) as a test case. The S&P 500 is the stock market index that tracks the performance of the 500 largest companies in the United States. The goal of this case study is to successfully build a machine learning pipeline that tries to predict the next day’s price direction of SPY while tackling the problems that come along the way.

2 Data

2.1 Source

Using Yahoo Finance, the daily Open, High, Low, Closing, and Volume data for SPY were retrieved. The data was pulled from January 1, 2015, to January 1, 2025. Exactly 10 years of trading data were used. Any weekends, holidays, or any other days when the stock market was closed were ignored. There were a total of 2516 raw trading days.

2.2 Feature Engineering

Feature engineering, which some people consider the most important step in machine learning, was particularly hard to decide on. Initially, five basic features intended to capture the behaviour of the SPY were chosen:

- **Daily Return:** The percentage change in closing price from the previous trading day,

$$\text{Return}_t = \left(\frac{\text{Close}_t - \text{Close}_{t-1}}{\text{Close}_{t-1}} \right) * 100. \quad (1)$$

- **5-Day and 20-Day Moving Averages (MA5, MA20):** A moving average is a calculation that takes the arithmetic mean of a given set of prices of an asset over a specific number of days in the past. We calculated two moving averages (5-day and 20-day).

$$\text{MA5} = \frac{\text{Close}_t + \text{Close}_{t-1} + \text{Close}_{t-2} + \text{Close}_{t-3} + \text{Close}_{t-4}}{5} \quad (2)$$

$$\text{MA20} = \frac{1}{20} \sum_{i=0}^{19} \text{Close}_{t-i} \quad (3)$$

- **10-Day Rolling Volatility:** The measure of how a stock price bounces up and down over a period of 10 Days.

$$\sigma_{10,t} = \sqrt{\frac{1}{9} \sum_{i=0}^9 (\text{Return}_{t-i} - \overline{\text{Return}}_{10,t})^2} \quad (4)$$

- **14-Day Relative Strength Index (RSI):** The 14-Day Relative Strength Index is intended to chart the current strength or weakness of an asset based on the closing price of a recent closing period, computed as

$$\text{RSI} = 100 - \frac{100}{1 + \text{RS}}, \quad (5)$$

$$\text{RS} = \frac{\text{Average Gain}_{14}}{\text{Average Loss}_{14}}, \quad (6)$$

$$\Delta \text{Close}_t = \text{Close}_t - \text{Close}_{t-1}, \quad (7)$$

$$G_t = \max(\Delta \text{Close}_t, 0), \quad (8)$$

$$L_t = \max(-\Delta \text{Close}_t, 0), \quad (9)$$

$$\text{Average Gain}_{14} = \frac{1}{14} \sum_{i=0}^{13} G_{t-i}, \quad (10)$$

$$\text{Average Loss}_{14} = \frac{1}{14} \sum_{i=0}^{13} L_{t-i}. \quad (11)$$

It can be noticed that RSI is algebraically a rescaled sigmoid function. Substituting $x = \ln(RS)$ into the standard sigmoid $\sigma(x) = \frac{1}{1+e^{-x}}$ gives $\sigma(\ln(RS)) = \frac{RS}{1+RS}$, which is $RSI/100$. This neat little fact is important to us because it shows how RSI uses a sigmoid function to squish an infinite ratio of gain-to-loss into a comprehensible scale of 0 to 100.

Rolling-window features require a warm-up period before they are defined (e.g., MA20 needs 20 prior days of history), so rows with any missing feature value were dropped. The total number of rows is now reduced to 2,497 rows.

2.3 Target Definition

To calculate the target, a simple question was asked. Was the price higher than yesterday? If the answer was yes, the Target was assigned a value of 1. If not, it would be assigned a value of 0. Expressing it as a piecewise function looks like this:

$$\text{Target}_t = \begin{cases} 1 & \text{if } \text{Close}_{t+1} > \text{Close}_t \\ 0 & \text{if } \text{Close}_{t+1} \leq \text{Close}_t \end{cases} \quad (12)$$

2.4 Train/Test Split

As we are dealing with time series data, a randomly selected train/test split wouldn't work. It would cause data to be leaked from the test period into the training period because the rolling windows would overlap. Instead, a chronological split was used. All the data from January 1, 2015 to January 1, 2023 was used to train the model, and the data from January 1, 2023 to January 1, 2025 was used to test the model. This split approximately comes out to be 80/20.

3 Methodology

3.1 Baseline Model

The model of choice here was Logistic Regression because we are dealing with binary targets. It suits the problem well. Logistic Regression is easy to interpret, and the output is naturally a probability in $[0, 1]$ obtained using the sigmoid function. The model was trained on the 5 chosen features using an unweighted loss.

For each trading day t , the model first computes a linear combination of the features, known as the logit:

$$z_t = \beta_0 + \beta_1 \cdot \text{Return}_t + \beta_2 \cdot \text{MA5}_t + \beta_3 \cdot \text{MA20}_t + \beta_4 \cdot \text{Volatility10}_t + \beta_5 \cdot \text{RSI14}_t \quad (13)$$

This logit is then passed through the sigmoid function to produce a probability:

$$P(\text{Target}_t = 1 \mid X_t) = \sigma(z_t) = \frac{1}{1 + e^{-z_t}} \quad (14)$$

Using Equation 13 in Equation 14 gives the full model, expressed directly in terms of the five engineered features:

$$P(\text{Target}_t = 1 \mid X_t) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 \cdot \text{Return}_t + \beta_2 \cdot \text{MA5}_t + \beta_3 \cdot \text{MA20}_t + \beta_4 \cdot \text{Volatility10}_t + \beta_5 \cdot \text{RSI14}_t)}} \quad (15)$$

and this probability is converted into a discrete prediction by using 0.5 as the threshold:

$$\hat{y}_t = \begin{cases} 1 & \text{if } P(\text{Target}_t = 1 \mid X_t) \geq 0.5 \\ 0 & \text{if } P(\text{Target}_t = 1 \mid X_t) < 0.5 \end{cases} \quad (16)$$

Table 1 summarizes the notation used in Equations 13–16.

Table 1: Notation used in the Logistic Regression model specification.

Symbol	Meaning
t	Index of a trading day (a row in the dataset)
Return_t	Daily return on day t
MA5_t	5-day moving average of closing price on day t
MA20_t	20-day moving average of closing price on day t
Volatility10_t	10-day rolling volatility (std. dev. of returns) on day t
RSI14_t	14-day Relative Strength Index on day t
β_0	Intercept (bias) term learned during training
$\beta_1, \dots, \beta_5$	Learned weights (coefficients), one per feature
z_t	Linear combination of features on day t (the “logit”)
$\sigma(x)$	Sigmoid function, $\sigma(x) = \frac{1}{1+e^{-x}}$
$P(\text{Target}_t = 1 \mid X_t)$	Model’s predicted probability that day t is followed by an up-day
$\hat{y}_t$	Final discrete prediction for day t (0 = Down, 1 = Up)

3.2 Naive Baseline

To evaluate the accuracy of the model, an always up-predicting model was created to compare the performance. Since SPY has steadily charted up during these 10 years, it is a difficult benchmark to beat. There might have been periods of massive drops, but overall SPY has gone up.

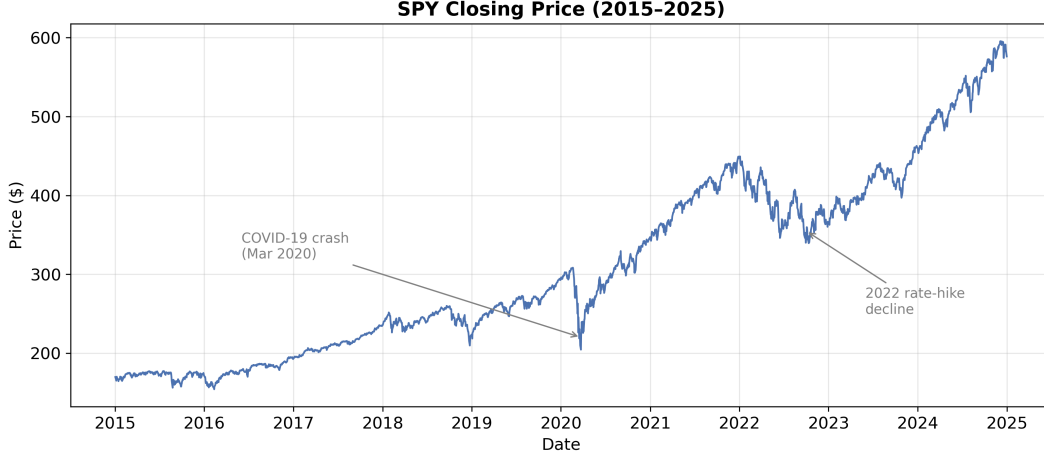


Figure 1: SPY closing price, 2015–2025. The series shows an upward trend, interrupted by the COVID-19 crash (March 2020) and a slower decline through 2022.

3.3 Class-Weight Balancing

Since the number of up-days(57.4%) is more than the number of down-days, a model can simply try to always predict up to get accurate to a certain degree. So, in actuality the model is not learning anything. This can be fixed by adding a factor that converts the unbalanced class weight to a balanced class weight. For this, we have to look at the loss function, which the model is trying to minimize:

$$F(\beta) = -\frac{1}{n} \sum_{i=1}^n \left[y_i \log(p_i) + (1 - y_i) \log(1 - p_i) \right] \quad (17)$$

where $p_i = P(\text{Target}_i = 1 \mid X_i)$ is the model's predicted probability for training sample i , and $y_i \in \{0, 1\}$.

The factor for class c would look like this:

$$w_c = \frac{n}{K \cdot n_c} \quad (18)$$

where n is the total number of training samples, K is the number of classes ($K = 2$ here), and n_c is the number of training samples belonging to class c .

Since there are fewer down days, $n_0 < n_1$ and hence $w_0 > w_1$. This will cause the penalty to be higher if we misclassify a down-day compared to when we misclassify an up-day. When we incorporate this factor to the loss function, it would look like this:

$$F_{\text{weighted}}(\beta) = -\frac{1}{n} \sum_{i=1}^n w_{y_i} \left[y_i \log(p_i) + (1 - y_i) \log(1 - p_i) \right] \quad (19)$$

where w_{y_i} is the weight corresponding to sample i 's class. Each misclassification is now scaled by the weight of its own class, which forces the optimizer to reduce errors on both classes rather than defaulting to choosing the majority.

Table 2 summarizes the additional notation introduced in Equations 17–19.

Table 2: Additional notation used in the class-weighted loss function.

Symbol	Meaning
n	Total number of training samples
i	Index of an individual training sample
y_i	True label of training sample i (0 = Down, 1 = Up)
p_i	Model's predicted probability that sample i is class 1
K	Number of classes ($K = 2$)
c	A class label ($c \in \{0, 1\}$)
n_c	Number of training samples belonging to class c
w_c	Balanced weight assigned to class c
w_{y_i}	Weight applied to sample i , based on its true class y_i
$F(\beta)$	Standard (unweighted) log-loss
$F_{\text{weighted}}(\beta)$	Class-weighted log-loss

4 Results

4.1 Baseline Logistic Regression

The unweighted Logistic Regression model achieved an accuracy of 55.6% on the test set. This result is less than the accuracy of the always up-predicting model, which comes out to be 57.4%. Table 3 and Figure 2 show the full classification results.

Table 3: Baseline Logistic Regression classification report.

Class	Precision	Recall	F1-score	Support
0 (Down)	0.44	0.15	0.23	214
1 (Up)	0.58	0.85	0.69	288
Accuracy	0.56			502

Table 4: Explanation of classification metrics used to evaluate model performance.

Metric	Description
Precision	Of all the samples the model predicted as a given class, the fraction that were actually correct.
Recall	Of all the samples that actually belong to a given class, the fraction the model correctly identified.
F1-score	The harmonic mean of precision and recall, combining both into a single number. Drops sharply if either precision or recall is low, making it a stricter measure than a simple average.
Support	The number of actual samples belonging to that class in the test set, independent of how the model performed.

Confusion Matrix — Baseline Logistic Regression

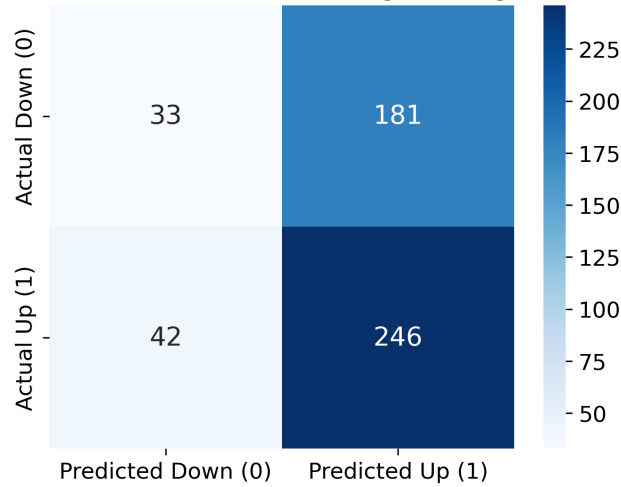


Figure 2: Confusion matrix for the baseline Logistic Regression model.

The recall on down-days is only 0.15, meaning the model correctly identified just 15% of actual down-days. Of the 502 test predictions, 427 (85%) were up-days, which shows the model leaned heavily towards the majority class rather than actually learning anything.

4.2 Class-weighted Logistic Regression

Applying weights to the class significantly shifted the model’s behavior. The accuracy decreased to 42.0%, but the behavior of the model was inverted. Recall on down-days rose to 0.95, while recall on up-days collapsed to 0.03.

Table 5: Class-weighted Logistic Regression classification report.

Class	Precision	Recall	F1-score	Support
0 (Down)	0.42	0.95	0.58	214
1 (Up)	0.42	0.03	0.05	288
Accuracy	0.42			502

Confusion Matrix — Class-Weighted Logistic Regression

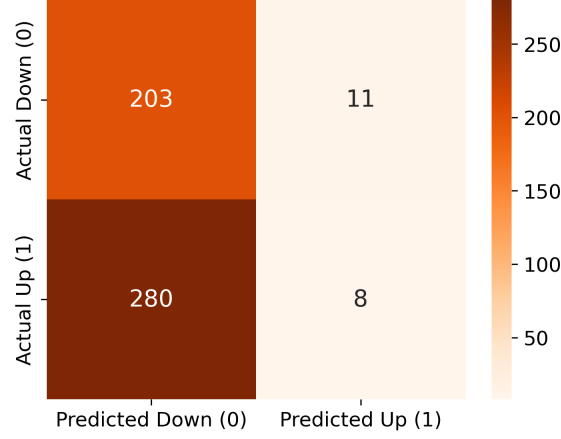


Figure 3: Confusion matrix for the class-weighted Logistic Regression model.

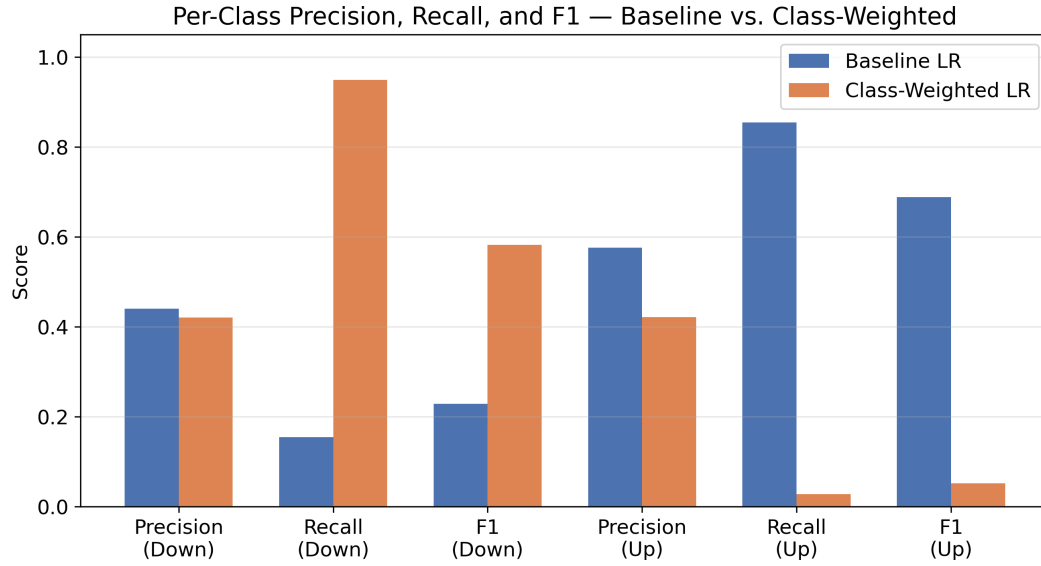


Figure 4: Per-class precision, recall, and F1-score for the baseline versus class-weighted models.

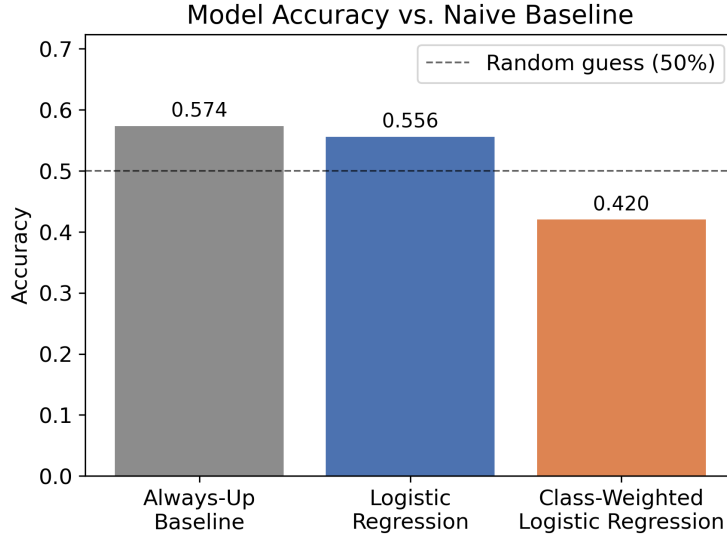


Figure 5: Overall test accuracy of the always-up model, the baseline Logistic Regression model, and the class-weighted Logistic Regression model.

5 Discussion

The results of the experiment provide information that is subtle but is the most important thing we need to look at. The class-weight balancing only addresses a bias that occurs during training. It stops the model from taking the shortcut that it does by favoring the majority class. But it cannot catch a pattern or a signal that is not present in the feature set. In other words, neither of the two models learned any significant details that would allow us to accurately predict the next day's up or down direction of SPY.

The results that we found are also consistent with the weak form of the Efficient Market Hypothesis. It tells us that historical price data is incorporated in the current price for a heavily traded, highly liquid asset like the SPY. And that we cannot use historical data to try to predict future asset price movement.

6 Conclusion

This case study provided a complete machine learning pipeline for predicting the next-day price direction of SPY. The model used 5 technical features and Logistic Regression. The baseline model underperformed an always-up predicting model by a small margin. After applying class weight balancing, the bias towards the majority class was fixed, but it did not reveal any predictive signals. It rather shifted the bias in the other direction. The results that were found are consistent with the difficulty in predicting the stock market. The model can be improved by using richer feature sets, non-linear models, and profitability-based evaluation.

A Code Listing

The following code implements the full pipeline described in this paper.

```
1 import yfinance as yf
2 import pandas as pd
3
4 data = yf.download("SPY", start="2015-01-01", end="2025-01-01")
5
6 data.columns = data.columns.get_level_values(0)
```

Listing 1: Data collection and preprocessing

```
1 # Daily return
2 data['Return'] = data['Close'].pct_change()
3
4 # Moving averages
5 data['MA5'] = data['Close'].rolling(window=5).mean()
6 data['MA20'] = data['Close'].rolling(window=20).mean()
7
8 # Rolling volatility
9 data['Volatility10'] = data['Return'].rolling(window=10).std()
10
11 # RSI (14-day)
12 delta = data['Close'].diff()
13 gain = delta.clip(lower=0)
14 loss = -delta.clip(upper=0)
15
16 avg_gain = gain.rolling(window=14).mean()
17 avg_loss = loss.rolling(window=14).mean()
18
19 rs = avg_gain / avg_loss
20 data['RSI14'] = 100 - (100 / (1 + rs))
```

Listing 2: Feature engineering: return, moving averages, volatility, RSI

```
1 # Target
2 data['Target'] = (data['Close'].shift(-1) > data['Close']).astype(int)
3
4 data_clean = data.dropna()
```

Listing 3: Target construction and NaN cleanup

```
1 split_date = "2023-01-01"
2
3 train = data_clean[data_clean.index < split_date]
4 test = data_clean[data_clean.index >= split_date]
5
6 features = ['Return', 'MA5', 'MA20', 'Volatility10', 'RSI14']
7
8 X_train = train[features]
9 y_train = train['Target']
10 X_test = test[features]
11 y_test = test['Target']
```

Listing 4: Chronological train/test split

```

1 from sklearn.linear_model import LogisticRegression
2 from sklearn.metrics import accuracy_score, confusion_matrix,
  classification_report
3
4 model = LogisticRegression()
5 model.fit(X_train, y_train)
6
7 predictions = model.predict(X_test)
8 accuracy = accuracy_score(y_test, predictions)
9 print("Model accuracy:", accuracy)
10
11 baseline_accuracy = y_test.mean()
12 print("Always-up baseline accuracy:", baseline_accuracy)
13
14 print(confusion_matrix(y_test, predictions))
15 print(classification_report(y_test, predictions))

```

Listing 5: Baseline Logistic Regression and naive baseline

```

1 model_balanced = LogisticRegression(class_weight='balanced')
2 model_balanced.fit(X_train, y_train)
3
4 predictions_balanced = model_balanced.predict(X_test)
5
6 print("Balanced model accuracy:", accuracy_score(y_test,
  predictions_balanced))
7 print(confusion_matrix(y_test, predictions_balanced))
8 print(classification_report(y_test, predictions_balanced))

```

Listing 6: Class-weight-balanced Logistic Regression